

Relatório Técnico — Shell para Sistemas Baseados em Conhecimento

Disciplina: Inteligência Artificial — 2026.1 · **Lista 1 (AB2), Questão 1 Tema:** Ferramenta genérica (shell) para construção de sistemas especialistas de diagnóstico e recomendação, nos moldes do *Expert SINTA*. **Aplicação demonstrativa:** Suporte Técnico de Computadores.

1. Visão geral

Foi desenvolvida uma **ferramenta genérica (shell)** que implementa a arquitetura conceitual de um **agente baseado em conhecimento**. A ferramenta é totalmente **independente de domínio**: todo o conhecimento é declarativo e fica em um arquivo de base de conhecimento (JSON). Para criar uma nova aplicação, o especialista apenas escreve a base — **sem alterar uma única linha do código-fonte da shell**. Isso é comprovado pelo script `demo_reuso.py`, que aplica a mesma shell a um segundo domínio (diagnóstico de pragas agrícolas).

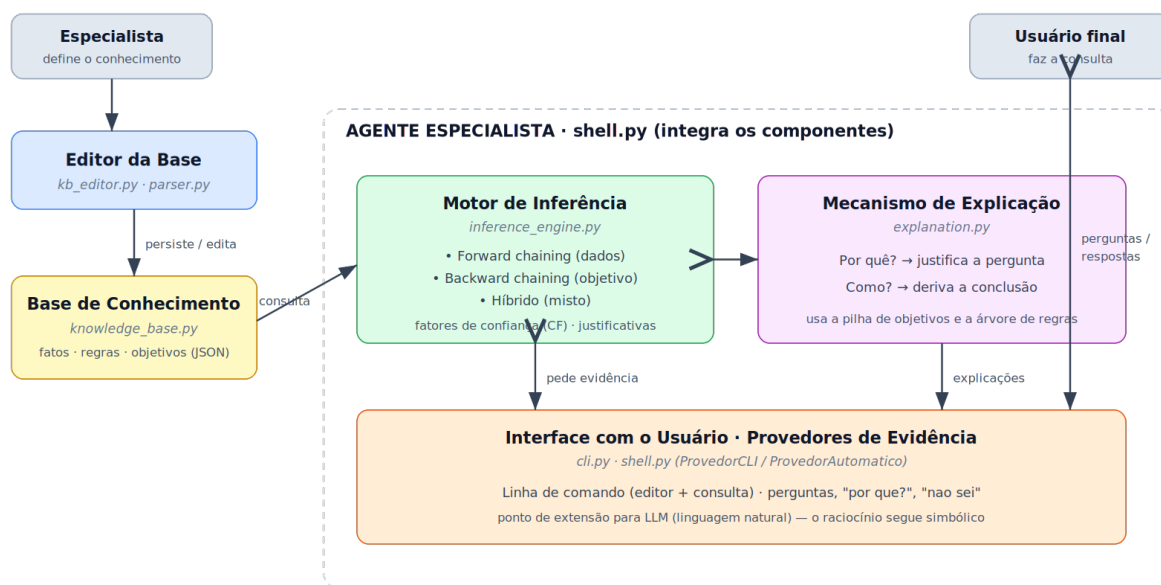
A shell cumpre os cinco requisitos centrais da questão: construir bases de fatos e regras de produção; realizar diagnósticos; recomendar ações/tratamentos; explicar o raciocínio; e ser reutilizável em diferentes domínios.

2. Arquitetura implementada

A solução segue a arquitetura clássica de um sistema especialista, com os módulos exigidos no enunciado, organizados no pacote `expert_shell`:

Arquitetura da Shell — Agente Baseado em Conhecimento

Diagnóstico e recomendação · genérica e reutilizável (o conhecimento é declarativo)



Fluxo: o especialista escreve a base → o usuário consulta → o motor infere com base nas regras → a interface coleta evidências → a explicação justifica.

Reuso: trocar de domínio = trocar o arquivo da base de conhecimento. O código da shell não muda.

Módulo	Arquivo	Responsabilidade
Editor da Base	kb_editor.py	CRUD de fatos/regras a partir de texto; persistência.
Base de Conhecimento	knowledge_base.py	Estruturas Variável, Regra, BaseConhecimento; (de)serialização JSON.
Parser	parser.py	Traduz SE ... E ... ENTÃO ... ↔ objeto Regra.
Motor de Inferência	inference_engine.py	Forward, backward e híbrido; fatores de confiança; justificativas.
Explicação	explanation.py	Responde "Por quê?" e "Como?".
Shell/Agente	shell.py	Integra os componentes; provedores de evidência.
Interface	cli.py	Menu de edição e de consulta em linha de comando.

3. Estrutura de representação do conhecimento

O conhecimento é representado por **três elementos declarativos**:

Variáveis (Variável) — cada atributo do domínio. Possuem nome, texto da pergunta, lista de valores possíveis e o atributo perguntável:

- *perguntáveis* → o valor é uma **evidência** obtida do usuário (ex.: liga, tela);
- *inferíveis* → o valor é **deduzido por regras** (ex.: diagnostico, recomendacao).

A própria shell marca como inferível qualquer variável que apareça em alguma conclusão.

Regras de produção (Regra) — no formato exigido SE condição1 E condição2 E ... ENTÃO conclusão, onde cada condição/conclusão é um par variável = valor. Cada regra carrega um **fator de confiança (CF** □ **[-100, 100])** para modelar incerteza, no estilo do Expert SINTA/MYCIN.

Objetivos — variáveis-alvo que o motor tenta resolver (as hipóteses de diagnóstico). Na aplicação demonstrativa: ["diagnostico", "recomendacao"].

Toda a base é serializada em **JSON**, atendendo ao requisito de persistência. Exemplo de regra no formato textual aceito pelo editor:

```
SE temperatura = alta E ventoinha = parada ENTÃO diagnostico = superaquecimento CF 90
```

4. Estratégia de inferência

O motor implementa **obrigatoriamente os três modos** pedidos.

Encadeamento para frente (Forward / dirigido pelos dados). Coleta as evidências e dispara repetidamente todas as regras cujas condições já estejam satisfeitas, acrescentando novos fatos até não haver mais mudanças (ponto fixo). Bom para "dado tudo o que sei, o que se conclui?".

Encadeamento para trás (Backward / dirigido ao objetivo). Para cada objetivo, localiza as regras que o concluem e tenta **provar** suas condições recursivamente: se a variável da condição é perguntável e desconhecida, **pergunta ao usuário**; se é inferível, tenta prová-la por outras regras. Duas economias de perguntas atuam aqui: (i) **poda por condição** — as condições de uma regra são avaliadas da esquerda para a direita e, na primeira que falha, as demais nem chegam a ser perguntadas; (ii) **parada antecipada** — assim que algum valor do objetivo atinge um CF igual ou superior ao limiar de parada (parâmetro `limiar_parada`, padrão 85), o motor deixa de avaliar as regras restantes daquele objetivo. Resultado medido na base demonstrativa: no cenário "não liga + cheiro de queimado" o backward fecha o diagnóstico com **2 perguntas**, e no cenário de malware com **7**, contra as **17** do forward (que sempre coleta toda a evidência). Para obter o **ranking exaustivo** de todas as hipóteses concorrentes, basta usar `limiar_parada=None`.

Estratégia híbrida (mista). (1) Executa um **forward de consolidação** sobre os fatos já conhecidos, sem fazer novas perguntas, deduzindo tudo o que for possível; em seguida (2) faz **backward** sobre os objetivos (também com parada antecipada) para preencher apenas as lacunas restantes. Combina o aproveitamento do forward com o foco do backward.

Tratamento de incerteza (fatores de confiança). O CF do antecedente de uma regra é o **mínimo** dos CFs das condições; o CF da conclusão é $CF_{\text{antecedente}} \times CF_{\text{regra}} / 100$. Quando várias regras concluem o mesmo fato, os CFs são combinados pela fórmula de **MYCIN** (`combinar_cf`). Assim, evidências convergentes **reforçam** a hipótese — em modo exaustivo, no cenário de RAM, as regras R3 (85) e R4 (92) elevam a confiança para ~99. Regras com CF abaixo do limiar (`LIMIAR_CF = 20`) não disparam. Observação: com a parada antecipada ligada, o diagnóstico pode fechar já na primeira regra confiante (ex.: malware com CF 85 por R8), trocando um pouco de corroboração por bem menos perguntas — um compromisso típico de shells de diagnóstico, ajustável pelo `limiar_parada`.

O resultado de uma consulta é um **ranking de hipóteses** por confiança, atendendo ao requisito de "exibir a hipótese mais provável" e lidar com hipóteses concorrentes.

5. Mecanismo de explicação

Implementado em `explanation.py`, no estilo dos sistemas especialistas clássicos, respondendo às duas perguntas exigidas:

Por quê? — Justifica por que uma pergunta foi feita. O motor mantém uma **pilha de objetivos** (`goal stack`): ao perguntar bipes enquanto avalia `diagnostico = memoria_ram` pela regra R3, o sistema responde:

Porque estou avaliando a hipótese 'diagnostico = memoria_ram'. A regra R3 exige a condição 'bipes = repetidos', e preciso conhecer 'bipes' para verificá-la.

Como? — Explica como uma conclusão foi obtida, percorrendo a **árvore de justificativas** registrada a cada disparo (qual regra, com quais fatos e qual CF), recursivamente até as evidências do usuário. Exemplo real de saída:

- 'diagnostico = memoria_ram' (CF=99) foi concluído pela regra R3 (CF da regra=85), porque:
 - 'liga = sim' valia (CF=100)
 - 'bipes = repetidos' valia (CF=100)
- 'diagnostico = memoria_ram' (CF=99) foi concluído pela regra R4 (CF da regra=92), porque:
 - 'liga = sim' valia (CF=100)
 - 'tela = preta' valia (CF=100)

- 'bipes = repetidos' valia (CF=100)

6. Interface com o usuário

A interface principal é em **linha de comando** (`cli.py`), com dois modos: o **Editor** (criar/listar/alterar/remover regras e variáveis, definir objetivos, salvar/carregar) e a **Consulta** (escolher a estratégia de inferência; durante as perguntas o usuário pode digitar *por que?* para obter a justificativa, ou *ou não sei*; ao final, pedir *como?* para qualquer conclusão).

A arquitetura desacopla o motor da interface por meio de **provedores de evidência** (`ProvedorCLI`, `ProvedorAutomatico`), o que permite trocar a interface (web, GUI) ou plugar a extensão com LLM sem mexer no motor.

Extensão com IA Generativa (opcional, implementada). A interface web inclui um botão *"Tornar natural (IA)"* que **reescreve as explicações "Por quê?" / "Como?"** em linguagem mais fluida usando um **LLM local via Ollama** (modelo padrão `llama3.2`, configurável por `OLLAMA_MODEL`). É **opcional e gratuito**: sem o Ollama rodando, o botão nem aparece e a shell funciona normalmente (degradação graciosa). **O raciocínio permanece simbólico** — o LLM não infere nada; apenas reformula um texto que o próprio motor de regras já gerou. O mesmo ponto de extensão (provedores de evidência + formatador de explicações) permitiria, no futuro, interpretar respostas em linguagem natural e convertê-las nos pares `variável = valor` esperados, sem tocar no motor.

7. Base de conhecimento demonstrativa

Domínio: **Suporte Técnico de Computadores** (`bases/suporte_tecnico.json`).

- **27 regras** (19 de diagnóstico + 8 de recomendação) — acima das 20 exigidas;
- **17 variáveis perguntáveis** com seus valores, totalizando **mais de 30 fatos**

possíveis** (pares `variável = valor`);

- **8 diagnósticos distintos** — acima das 5 hipóteses exigidas:

fonte defeituosa, memória RAM, superaquecimento, malware, falha de disco, problema do provedor, configuração de rede local e bateria degradada;

- cada diagnóstico encadeia uma **recomendação de ação**.

8. Exemplos de consultas realizadas

Resultados reproduzíveis com `python exemplos/demo_automatica.py`:

Cenário	Evidências principais	Modo	Diagnóstico (CF)	Recomendação	Perguntas
1	lentidão + pop-ups + programas estranhos	backward	malware (85)	antivírus + backup	7

Cenário	Evidências principais	Modo	Diagnóstico (CF)	Recomendação	Perguntas
2	esquenta muito + reinicia + poeira	forward	superaquecimento (95)	limpar + pasta térmica	17
3	HD com clique + erro de disco + lentidão	híbrido	falha de disco (95)	backup + trocar disco	10
4	não liga + cheiro de queimado	backward	fonte defeituosa (95)	trocar fonte	2
5	sem internet no PC e em outros aparelhos	backward	problema do provedor (90)	reiniciar roteador / provedor	11
6	tela preta + bipes repetidos	backward	memória RAM (85)	reassentar/testar RAM	4

A coluna **Perguntas** evidencia o efeito da parada antecipada do backward/híbrido frente ao forward (que sempre pergunta as 17 variáveis). Em modo exaustivo (`limiar_parada=None`) os CFs sobem por corroboração (ex.: malware chega a ~100 disparando R8, R9 e R10), ao custo de mais perguntas.

9. Limitações e possíveis melhorias

Limitações. (a) A lógica é proposicional (pares variável = valor), sem variáveis lógicas/quantificação como em Prolog. (b) Os fatores de confiança são uma heurística (MYCIN), não probabilidades calibradas. (c) Não há detecção de conflitos/loops entre regras além da proteção contra recursão. (d) A interface é textual.

Melhorias futuras. (a) Suporte a operadores nas condições (>, <, ≠, intervalos) e a conectivo OU; (b) verificação automática de consistência e de regras redundantes na edição; (c) interface web/gráfica reaproveitando os provedores de evidência; (d) a extensão com LLM descrita na Seção 6, mantendo o raciocínio simbólico; (e) aprendizado dos CFs a partir de casos históricos.

10. Como executar

Rode a partir da **raiz do projeto**. Requer apenas Python 3.8+ (biblioteca padrão).

```
# uso normal: consulta interativa de diagnóstico (ponto de entrada)
python main.py
# equivalente: python -m expert_shell.cli bases/suporte_tecnico.json

# exemplos não-interativos (opcionais)
python exemplos/demo_automatica.py          # 6 cenários nos 3 modos, com explicações
python exemplos/demo_reuso.py                # mesma shell em outro domínio
python exemplos/construir_base_suporte.py    # regenera bases/suporte_tecnico.json

# testes automatizados
python tests/test_expert_shell.py
```

No `main.py/CLI`: escolha 2 (consulta) → selecione a estratégia → responda às perguntas (valor pedido, ou por que?, ou não sei) → ao final veja o diagnóstico e peça como?. A opção 1 abre o

editor da base.

11. Mapa dos entregáveis

Entregável exigido	Onde está
Código-fonte da shell	pacote expert_shell/
Base de conhecimento da demonstração	bases/suporte_tecnico.json (+ bases/pragas_agricolas.json)
Relatório técnico	este documento (docs/RELATORIO_TECNICO.md)
Demonstração do funcionamento	exemplos/demo_automatica.py e a CLI interativa (main.py)